



Lesson 06: Backend Tools (1)

Front-end Integration

Để tích hợp hệ thống Backend với Frontend, lập trình viên Backend cần phải giao tiếp với lập trình viên Frontend:

Thống nhất cần có những API nào, thiết kế API một cách thích nghi và hiệu quả để Frontend có thể sử dụng đơn giản:

VD:

- Khi FE cần pagination, ta cần phải thêm các field như offset, limit trong request và dùng nó trên câu query SQL
- Khi FE cần infinite scrolling, thì response ngoài list item ra thì cũng phải có 1 cờ boolean hasNext để FE biết có còn scroll được nữa không
- Một vài API sẽ luôn được gọi chung với nhau thì BE có thể gộp 2 API lại thành 1 để FE chỉ cần gọi 1 http request
- Khi FE có tính năng sort theo 1 field nào đó, thì ta cũng phải pass field muốn sort, cũng như cách sort trong request (vì nếu để FE tự sort thì nó chỉ là sort trong trang pagination đó, chứ ko phải sort tổng thể toàn bộ data)
- Nếu update 1 item, thông thường API bên BE sẽ yêu cầu đầy đủ các field của item đó, nhưng FE đôi lúc chỉ muốn gửi 1 vài field bị đổi của item đó chứ không muốn gửi hết full data của item (vì collect lại full data của item có thể khá tốn công), thì ta có thể làm thêm 1 API tương tự nhưng với method là PATCH, và chỉ sửa những field mà FE đưa lên.
- ...

Tuy nhiên chúng ta cũng cần lưu ý nếu những thay đổi này làm ảnh hưởng tới performance hay logic của hệ thống hay không, nếu có thì cần phải tìm ra giải pháp hiệu quả cho cả đôi đường:

VD:

- FE có thể muốn data dưới client được update liên tục, nên sẽ gọi 1 API GET liên tục với 1 time ngắn (polling), việc này có thể tăng lượng **request vào BE rất nhiều** và làm chậm hệ thống. BE có thể yêu cầu đổi thành **long polling** hoặc websocket: sẽ chủ động gửi data về FE chỉ khi có data mới chứ không để FE polling liên tục. Việc này sẽ giúp giảm lượng request vô nghĩa lên BE ([polling vs long polling vs websocket](#))
- ...

API Documents: Swagger

Viết document cho các API của BE là rất quan trọng, việc này giúp lập trình viên FE hiểu API và sử dụng nó đúng cách, đúng format.

Swagger là một tool hữu ích để tự động generate document cho HTTP API, vừa tiết kiệm thời gian cho BE, và FE cũng dễ đọc và sử dụng thử trực tiếp được các API này

Swagger không chỉ phổ biến với Golang, mà các ngôn ngữ khác đều có hỗ trợ, và cũng rất quen thuộc với lập trình viên FE, đây giống như ngôn ngữ chung cho FE và BE

Document:

- <https://github.com/swaggo/swag>
- <https://santoshk.dev/posts/2022/how-to-integrate-swagger-ui-in-go-backend-gin-edition/>

Install:

```
go install github.com/swaggo/swag/cmd/swag@latest
```

Tạo documents từ code có sẵn:

```
swag init -g cmd/web_app/main.go
```

Cách swagger generate document:

- Khi chạy "swag init" swagger sẽ scan hết các file .go và lấy tất cả các **godoc comment** ở trước các function của mỗi API, và tạo ra các file swagger.json và

swagger.yaml chứa thông tin các API mà service hỗ trợ.

- Ngoài ra swagger còn *đọc các struct mà ta define để show ra API* của ta cần request body như thế nào và sẽ trả về response như thế nào.

Swagger

- Phần godoc trong file main sẽ cung cấp các thông tin cơ bản của service, như:
 - Tên service
 - Description
 - Author
 - Base path cho mọi api
 - Authenticate trước khi gọi api

```
// @title      Gin Social network Service
// @version    1.0
// @description A simple social network management service API in Go using Gin framework.
// @termsOfService

// @contact.name   Dong Truong
// @contact.url    https://www.linkedin.com/in/dong-truong-56297a1d5/
// @contact.email  tpdongcs@gmail.com

// @license.name   Apache 2.0
// @license.url    http://www.apache.org/licenses/LICENSE-2.0.html

// @host          localhost:8080
// @basePath      /api/v1
// @securitydefinitions.oauth2.password OAuth2Password
// @tokenurl      https://example.com/oauth/token
// @scope.read    Grants read access
// @scope.write   Grants write access
// @scope.admin   Grants read and write access to administrative information

// @securitydefinitions.oauth2.accessCode OAuth2AccessCode
// @tokenurl      https://example.com/oauth/token
// @authorizationurl https://example.com/oauth/authorize
// @scope.admin   Grants read and write access to administrative information

func main() {
    flag.Parse()
    conf, err := configs.GetWebConfig(*path)
    fat.Println(conf)
    if err != nil {
        log.Fatalf("failed to parse config: %s", err)
    }
}
```

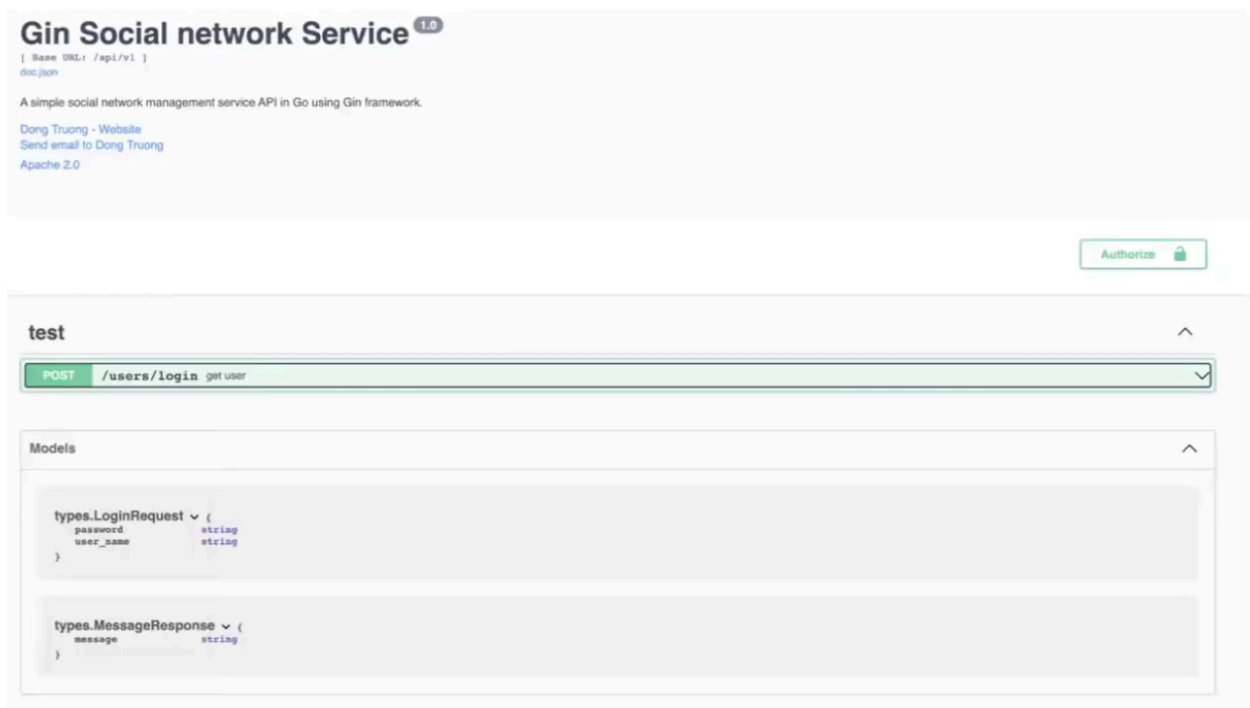
- Phần godoc trước mỗi api handler sẽ thể hiện cho mỗi api, bao gồm:
 - Tên
 - Method
 - Path
 - Description
 - Content-type
 - Request param (kiểu dữ liệu, và là param path, query hay body)
 - Response: Các status code có thể có, và body trả về của mỗi

```
// CheckUserNameAndPassword godoc
//
// @Summary      get user
// @Description   check user user_name and password
// @Tags         test
// @Accept       json
// @Produce      json
// @Param        request body types.LoginRequest true "login param"
// @Success      200 {object} types.MessageResponse
// @Failure      400 {object} types.MessageResponse
// @Failure      500 {object} types.MessageResponse
// @Router       /users/login [post]
func (svc *WebService) CheckUserNameAndPassword(ctx *gin.Context) {
    var jsonRequest types.LoginRequest
    err := ctx.ShouldBindJSON(&jsonRequest)
    if err != nil {
        ctx.JSON(http.StatusBadRequest, &types.MessageResponse{Message: err.Error()})
        return
    }
}
```

status

- Sau khi chạy "swag init", các file docs.go, swagger.yml và swagger.json sẽ được tạo ra trong folder docs
- Để có thể dựng một trang swagger UI trên chính service đó, ta chỉ cần import file docs.go ở trên và add handler của swagger và chạy service

```
import (  
    _ "github.com/EngineerPro0rg/BE-K01/docs"  
    swaggerFiles "github.com/swaggo/files"  
    ginSwagger "github.com/swaggo/gin-swagger"  
)  
  
func (c WebController) Run() {  
    r := gin.Default()  
    r.GET("/swagger/*any",  
        ginSwagger.WrapHandler(swaggerFiles.Handler))  
}
```



Responses	
Code	Description
200	OK
Example Value Model	
<pre>{ "message": "string" }</pre>	
400	Bad Request
Example Value Model	
<pre>{ "message": "string" }</pre>	
500	Internal Server Error
Example Value Model	
<pre>{ "message": "string" }</pre>	

PPROF (Profiling)

Khi hệ thống được viết bởi golang có các vấn đề về CPU, memory leak, goroutine leak, hay GC, pprof là một tool mà golang hỗ trợ nhằm giúp chúng ta tìm hiểu lý do và giải quyết vấn đề ([pprof docs](#))

Các tool pprof hỗ trợ:

- **goroutine** - stack traces of all current goroutines (3rd priority)
- **heap** - a sampling of all heap allocations (2nd priority)
- **threadcreate** - stack traces that led to the creation of new OS threads
- **block** - stack traces that led to blocking on synchronization primitives
- **mutex** - stack traces of holders of contended mutexes
- **profile** - cpu usage percentage on each part of code (1st priority)
- **trace** - cpu trace on each core

- **50 phút:** dùng ở function B
- **1 giây:** dùng ở function C

Ví dụ 1: Optimize thẳng A

- 5ph → 1ph (giảm 4 phút)
- Tổng thời gian: 1h → 56ph

Ví dụ 2: Optimize thẳng B

- 50ph → 25ph (giảm 25 phút)
- Tổng thời gian: 1h → 35ph

PPROF: Install on service

```
import (  
    "net/http/pprof"  
)  
  
r.GET("/debug/pprof/:param", func(context *gin.Context) {  
    pprof.Index(context.Writer, context.Request)  
})  
  
r.GET("/debug/pprof/profile", func(context *gin.Context) {  
    pprof.Profile(context.Writer, context.Request)  
})  
  
r.GET("/debug/pprof/trace", func(context *gin.Context) {  
    pprof.Trace(context.Writer, context.Request)  
})
```

PPROF: Get prof analysis data

There are 7 places you can get profiles in the default webserver: the ones mentioned above

- <http://localhost:6060/debug/pprof/goroutine>
- <http://localhost:6060/debug/pprof/heap>
- <http://localhost:6060/debug/pprof/threadcreate>
- <http://localhost:6060/debug/pprof/block>
- <http://localhost:6060/debug/pprof/mutex>

and also 2 more: the CPU profile and the CPU trace.

- <http://localhost:6060/debug/pprof/profile>
- <http://localhost:6060/debug/pprof/trace?seconds=5>

Tải file pprof từ các đường link này và sử dụng `go tool pprof -http=:6060 pprof-file.out` để trực quan hoá dữ liệu

Profiling là gì? CPU hoạt động như thế nào?

- Nếu bạn bật CPU profiling, `start_time`; `end_time`
 - **bật:** bạn muốn collect data profile ứng dụng của bạn trong khoảng thời gian nào
 - **ví dụ:** bật trong 9:00AM – 10:00AM
- CPU profiling, cứ mỗi 1s, nó sẽ stop chương trình của bạn, capture toàn bộ stack trace
- **Stack trace:** ở thời điểm t, show ra là function nào đang được CPU chạy
 - t = 1s: function A
 - t = 2s: function B
 - t = 3s: function A
 - t = 4s: function C
 - t = 5s: function A

- ⇒ Tổng hợp lại toàn bộ data theo các function như sau:

Function	Số snapshot
A	3 snapshot
B	1 snapshot
C	1 snapshot

⇒ Trong 1 tiếng đó, thống kê được:

function A take 3/5 time run the program

- Có logic function A và đoạn gọi function B

function B take 1/5 time run the program

function C take 1/5 time run the program

PPROF: Analysis CPU usage and bottleneck

- Từ những thông tin lấy được từ data được hiển thị, ta có thể biết được phần nào trong code đang sử dụng bao nhiêu resource CPU và có thể tìm cách để tối ưu những phần đó.
- Profile tool có thể xác định được một vài thành phần sử dụng nhiều CPU hơn ta expect.
- Hoặc nếu CPU usage cho Garbage Collector quá nhiều so với phần còn lại của service, có thể setup GC của chúng ta chưa tốt, hoặc ta sinh ra quá nhiều object không cần thiết.



Profiling gây hệ thống chậm đi quá nhiều. Chính vì vậy profile trong time ngắn thôi.

- Profile CPU 60s max

Cách profile đúng là sao

- Ứng dụng của bạn chạy bình thường thì ngon. Nhưng mà cứ khi có QPS > 100, ứng dụng bị đứng.

Q: làm sao bạn detect cái lỗi này bằng pprof

Option 1: bật profiling lúc nào cũng được

- Chạy pprof 60s
 - FuncA → 50% CPU
 - FuncX → 3% CPU

Option 2: step stress test chạy từ 9:00 – 9:10 (QPS > 100) (Tất nhiên là chỉ profile khi QPS > 100)

- Chạy pprof 60s trong khoảng 9:01 – 9:02
 - FuncX → 40% CPU

→ Homework

Ứng dụng stress test qua 1k QPS

Requirement: 10k QPS

→ Resolve bottle neck bằng pprof

PPROF: Analysis Memory usage and memory leak

- UI của memory heap tool cũng tương tự với profile tool, ta có thể xem những thành phần nào đang sử dụng bao nhiêu memory, hoặc bao nhiêu object được sinh ra từ phần đấy của code tại thời điểm lấy pprof file.
- Heap tool sẽ giúp chúng ta xác định được những thành phần nào sử dụng quá nhiều memory so với expect, hay sinh ra quá nhiều object không cần thiết. Và

từ đó tìm cách tối ưu bộ nhớ tại phần đấy.

- Memory leak là vấn đề ta thường gặp phải khi memory của process tăng lên liên tục và không có dấu hiệu giảm, GC không thể xóa bớt được các object đấy, và process sẽ chết nếu hết memory.
- Để tìm ra vị trí tạo memory leak, ta có thể lấy heap pprof tại nhiều thời điểm, và so sánh thành phần nào đòi cung cấp memory nhiều hơn sau 2 thời điểm đó, và tìm cách khắc phục.

```
( go tool pprof -http=:8080 --base dumps/heap-profile-cmp-001.pb.gz dumps/heap-profile-cmp-002.pb.gz )
```

Memory profiling hoạt động như thế nào

- Cứ mỗi khi bạn allocate 1 cái object → record bằng cách lưu lại ở đâu đó
 - **FuncA**: allocate 100 object, 100MB
- Cứ mỗi khi bạn free cái object → record bằng cách lưu lại ở đâu đó
 - **FuncA**: free 50 object, 50MB
- Hỏi là mỗi function đang dùng bao nhiêu memory ⇒ **function A: 50MB**

Golang đã làm sẵn cho bạn cái này, bạn chỉ cần bật lên là xong

- **memstat** ⇒ bật sẵn từ lúc application khởi động

Cách pprof

- Ở thời điểm bạn có thể hỏi golang là các function nào đang dùng bao nhiêu memory
 - **CPU**: Start_time, end_time
 - **Memory**:
 - Start_time = 0 (bắt đầu chương trình)
 - End_time: thời điểm hiện tại

Q: có pattern như sau: bạn thấy memory tăng ko ngừng, cứ 1 phút tăng 1MB

⇒ Làm sao biết chỗ nào bị memory leak

- có function nó tạo object nhưng mà không free objects?

Wrong:

1 **memstat** snap: 10:00

- funcA: 100MB
- funcB: 50MB

⇒ funcA dùng nhiều mem nên hãy optimize nó ⇒ **Wrong**

Right:

1 memstat snap: 10:05

- funcA: 110MB
- funcB: 55MB

1 memstat snap: 10:10

- funcA: 100MB
- funcB: 60MB

1 memstat snap: 10:20

- funcA: 110MB
- funcB: 70MB

⇒ detect thằng B nó có vấn đề, nó không free memory

Q: Nhưng mà sao chương trình vẫn biết là leak đó là của funcB, bởi vì B liên tục bị remove và thêm vào call stack?

Q: Vậy việc profiling memory này có làm chậm hệ thống?

⇒ Có, cực kỳ nhỏ nên ko đáng kể

PPROF

Note: Dữ liệu từ **pprof** không hẳn sẽ chỉ rõ vấn đề ta gặp phải, nhưng nó sẽ cho ta thấy những vị trí tiềm năng dễ tạo ra vấn đề. Vì vậy ta chỉ nên sử dụng pprof để gợi ý, và cần phải đọc và tìm hiểu rõ vấn đề và lý do cốt lõi tạo ra problem đang gặp phải.

Monitor system

Các hệ thống lớn và chạy trên live cần các tool giúp ta collect dữ liệu nhằm monitor sức khoẻ của hệ thống.

Có rất nhiều platform/ library giúp chúng ta làm những việc này như Prometheus, Elastic stack (APM), StatsD, chúng đều có chung những tính năng tương đồng nhau.

Ví dụ:

- Collect latency của các API, từng thành phần và dependency của hệ thống
- Collect error rate của các dependency
- Collect CPU, Memory usage hoặc status của các instance

Ngoài monitor hệ thống, các hệ thống này còn được dùng để đếm các logic mà hệ thống chạy, dữ liệu có thể được sử dụng để thống kê

Time series dataCPU (Gauge)

9:20: CPU: 50%

9:21: CPU: 60%

9:22: CPU: 55%

9:23: CPU: 70%

9:24: CPU: 65%

⇒ Vẽ thành các biểu đồ để dễ nhìn hơn

Error rate, QPS, Request Count (Counter)

9:20: Service A _ Error Count: 10

9:20: Service A _ Total Request Count: 100

⇒ Error_rate: 9:20 = 10% = 0.1

9:21: Service A _ Error Count: 5

9:21: Service A _ Total Request Count: 50

⇒ Error_rate: 9:20 = 10% = 0.1

Latency

9:20: Service A

- 3rd: 50ms
- 1st: 100ms
-
- 100th: 150ms
- 2nd: 200ms
- 4th: 300ms

Step 1: Sort cdc request theo thời gian xử lý



9:20:

- **p50 / avg / mean** (50 request có thời gian xử lý nhanh nhất) = [100ms] = thời gian của request thứ 50 sau khi sort
- **p90** (90 request có thời gian xử lý nhanh nhất) = 200ms
- **p99** (99 request có thời gian xử lý nhanh nhất) = 300ms (p99 là phổ biến nhất)

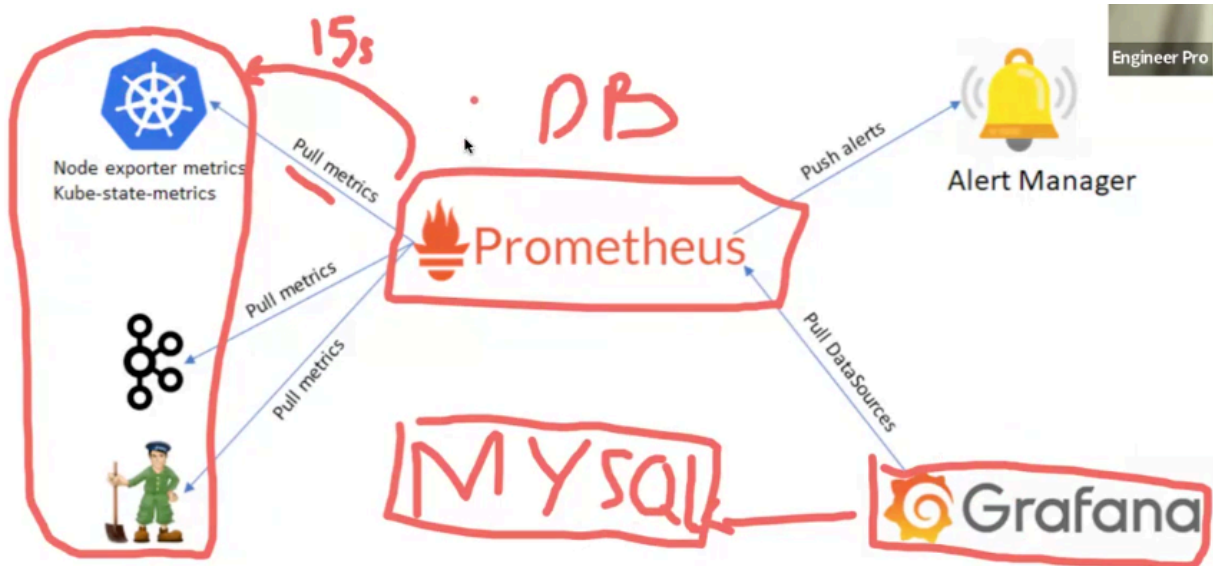
Prometheus

Là DB để lưu trữ time-series data

Cơ chế làm việc:

- Khi các service cài đặt prometheus, mỗi service instance, hoặc các hệ thống như kafka, zookeeper sẽ tạo 1 HTTP API nhằm cung cấp các thông tin metrics
- Một hệ thống tập trung sẽ có nhiệm vụ gọi vào tất cả các HTTP API sau mỗi vài giây để pull lượng dữ liệu này và lưu vào bộ nhớ
- Hệ thống này cũng sẽ cung cấp 1 ứng dụng web UI (thường là grafana) nhằm giúp ta visualize hoặc query những dữ liệu này từ bộ nhớ

- Ta cũng có thể dùng grafana để setup các alert để thông báo (gửi mail hoặc message ...) nếu hệ thống có vấn đề



- Cứ sau 1 khoảng tgian mà có thêm data thì gọi là time-series data
- pull metric hoặc app tự push metric vô DB

Setup prometheus cho các service:

```
import (
    "net/http"
    "github.com/prometheus/client_golang/prometheus/promhttp"
)

var prometheusHandler = promhttp.Handler()

r.GET("/metrics", func(context *gin.Context) {
    prometheusHandler.ServeHTTP(context.Writer, context.Request)
})
```

Init metrics lấy latency cho service

```

import (
    "github.com/prometheus/client_golang/prometheus"
    "github.com/prometheus/client_golang/prometheus/promauto"
)

var latencyExporter = promauto.NewSummaryVec(
    prometheus.SummaryOpts{
        Namespace: "be_k01", // name
        Subsystem: "webapp", // name
        Name: "latency",
        Help: "recall latency in milliseconds",
        Objectives: map[float64]float64{0.5: 0.05, 0.9: 0.01,
0.99: 0.001},
    },
    []string{"component", "status"},
)

```

Init metrics count cho service (đếm số lượng request, đếm error, đếm count số lần làm 1 logic nào đó ...)

```

import (
    "github.com/prometheus/client_golang/prometheus"
    "github.com/prometheus/client_golang/prometheus/promauto"
)

// be_k01_webapp_count
var countExporter = promauto.NewCounterVec(
    prometheus.CounterOpts{
        Namespace: "be_k01",
        Subsystem: "webapp",
        Name: "count",
        // -> be_k01_webapp_count
        Help: "recall count",
    },
)

```

```

    []string{"component", "type"},
)

```

```

func (svc *WebService) #GetUserNamePassword(ctx *gin.Context)
{
    start := time.Now()
    status := http.StatusOK
    countExporter.WithLabelValues("check user login", "total").Inc()

    defer func() {
        latencyExporter.WithLabelValues("check_user_login", strconv.Itoa(status)).Observe(float64(start.UnixMilli()))
    }()

    var jsonRequest types.LoginRequest
    err := ctx.ShouldBindJSON(&jsonRequest)
    if err != nil {
        countExporter.WithLabelValues("check_user_login", "bad_request").Inc()
        status = http.StatusBadRequest
        ctx.JSON(status, types.MessageResponse{Message: err.Error()})
        return
    }

    authentication, err := svc.authenticateAndPostClient.CheckUserAuthentication(ctx, &authen_and_post.UserInfo{
        UserName: jsonRequest.UserName,
        UserPassword: jsonRequest.Password,
    })
    if err != nil {
        countExporter.WithLabelValues("check user login", "call_api_failed").Inc()
        status = http.StatusInternalServerError
    }
}

```

```

        ctx.JSON(status, types.MessageResponse{Message: err.Error()})
    }
    return
}

if authentication.GetStatus() == authen_and_post.UserStatus_OK {
    countExporter.WithLabelValues("check user login", "success").Inc()
    ctx.JSON(status, types.MessageResponse{Message: "ok"})
    // change this later
    ctx.SetCookie("session_id", fmt.Sprintf("%d", authentication.Info.UserId), 0, "", "", false, false)
}
}

```

Khi hệ thống đang hoạt động, ta có thể lấy các thông số này bằng cách gọi vào `api/metrics`

<pre> 1646539457383 #current_timestamp_ms be_k0l_webapp_latency{component="check_user_login", status="200", quantile="0.99"} 120 be_k0l_webapp_latency{component="check_user_login", status="200", quantile="0.9"} 100 be_k0l_webapp_latency{component="check_user_login", status="200", quantile="0.5"} 70 be_k0l_webapp_count{component="check_user_login", type="total"} 1000 be_k0l_webapp_count{component="check_user_login", type="bad_request"} 17 be_k0l_webapp_count{component="check_user_login", type="call_api_failed"} 120 be_k0l_webapp_count{component="check_user_login", type="success"} 863 </pre>	- Các request chậm nhất (chiếm 1% lượng request) có latency > 120ms - Các request chậm nhất (chiếm 10% lượng request) có latency > 100ms - Các request chậm nhất (chiếm 50% lượng request) có latency > 70ms, đồng nghĩa với latency trung bình là 70ms - Tổng cộng đã có 1000 request check user login đã được gọi từ khi instance này dựng lên - Trong đó có 17 request bị bad request (1.7%) - 120 request bị lỗi ở bước gọi tới backend grpc (12%) - 863 request còn lại success (86.3%)
--	--

Hệ thống collect các metrics data (prometheus hoặc thanos) sẽ có nhiệm vụ gọi tới tất cả các HTTP API của các instances, và collect dữ liệu này sau mỗi 1 khoảng thời gian nhỏ (1s hoặc 5s)

Các dữ liệu như trên sẽ được thêm 1 label "instance" nhằm thể hiện data đó được lấy ở instance nào

```
global:
    scrape_interval: 5s

scrape_configs:
  - job_name: 'entrytask'
    static_configs:
      - targets:
        - 'host.docker.internal:19003'
```

- Lấy latency P99 trung bình của các instance (với 1 component nhất định và status là 200)

```
avg(be_k01_webapp_latency{component="check_user_login", status="200", quantile="0.99"})
```

- Lấy latency P99 trung bình của các instance (với 1 component nhất định, nhưng status là 4xx)

```
avg(be_k01_webapp_latency{component="check_user_login", status=~"4.*", quantile="0.99"})
```

- Nếu bạn chỉ muốn lấy của một instance duy nhất:

```
avg(be_k01_webapp_latency{component="check_user_login", status="200", quantile="0.99", instance="10.128.12.12:8081"})
```

- Lấy số lượng request trong 1 phút được gửi lên trên tất cả các instance

```
sum(increase(be_k01_webapp_count{component="check_user_login", type="total"}[1m]))
```

- Lấy số lượng request trong 1m được gửi lên trên tất cả các instance (hàm rate sẽ tự động đổi qua per second)

```
sum(rate(be_k01_webapp_count{component="check_user_login", type="total"}[1m]))
```

- Lấy tỷ lệ % error trên tổng số request:

```
sum(rate(be_k01_webapp_count{component="check_user_login", type!="success"}[1m])) /  
sum(rate(be_k01_webapp_count{component="check_user_login", type="total"}[1m])) * 100
```

Tất cả các câu query trên sẽ chỉ trả về 1 chuỗi time series duy nhất.

Nếu muốn lấy latency theo cả 3 quantile 0.99, 0.9 và 0.5 và cùng show trong 1 bảng chung để tiện nhìn và so sánh:

```
avg by (quantile)  
(be_k01_webapp_latency{component="check_user_login", status="200"})
```

Tracing

Trong môi trường microservice, làm thế nào để ta có thể trace một request sẽ hoạt động như thế nào qua từng service?

Thông thường trong hệ thống monolithic, tất cả các log sẽ được lưu vào file nằm trên cùng instance chạy service hiện tại.

Nhưng trong hệ thống microservice, vì một request phải đi qua nhiều service khác nhau, nên ta cần có một hệ thống chung để lưu tất cả các log của các service này.

Để group các log của các service khác nhau, mỗi request sẽ cần có request ID, và user ID để định danh cho một request (**request ID và user ID này sẽ được pass qua cho tất cả request qua tất cả các services được gọi qua**, và request ID thường sẽ được generate tại service đầu tiên được gọi từ FE)

Mỗi service khi muốn log ra một data log, cần phải đính kèm request ID và user ID này

Có 2 cách cơ bản để đưa log về một hệ cơ sở dữ liệu tập trung:

- Các service khi muốn bắn 1 đoạn log, sẽ gửi log này cho một message queue, hoặc một service nào đó để lưu.
- Các service sẽ viết vào một file local (tương tự các hệ thống monolithic thông thường) tuy nhiên mỗi container/instance cần phải cài một process riêng biệt

chạy song song (thường được gọi là log agent) để đọc file log local và sẽ gửi những đoạn log này cho message queue hoặc service log tập trung.

Khi các đoạn log này đã được thu thập và tập trung tại một nguồn (thường là một service hoặc file, hoặc hdfs) ta sẽ hỗ trợ developer có thể query/filter theo request ID, user ID.

Lúc đấy, tất cả các log data từ tất cả service sẽ được lấy ra theo request ID và user ID, nhằm giúp developer có thể tìm hiểu, debug, trace lỗi, cũng như logic của 1 request nhất định.